

Combining the Formal and the Informal

Jonathan Reicher Shachar Itzhaky

June 30, 2026

Problem Statement

In the intersection between the worlds of mathematical theory, formal logic, and programming, exists a species of tools called proof assistants. These are programming languages, meant not for building software, but for building mathematical proofs. Proof assistants, contrary to pen and paper, check formalized proofs for correctness and soundness.

These so called proof assistants admit only entirely formalized proofs, where every step is fully justified based on a complex logical semantics sitting on top of a complex declarative semantics. That is to say, not only are the logical steps themselves complex and rigorous, but so is the way you express your proof and how the steps are applied.

For many years, proof assistants were a niche, unpopular family of tools, used mostly by experts and enthusiasts. However, one member of that family in particular has been gaining traction in the mathematical community, bringing many inexperienced users to this world of expert-made, technical and rigorous proofs.

LEAN ([TODO: cite?](#)) is an interactive proof assistant geared towards a general math audience. Two things that make it especially appealing are its large ecosystem and its intuitive syntax. For example, it has a large library of formalized mathematics called `Mathlib`, which contains many foundational and advanced mathematical results and theories, and includes notation familiar to mathematicians. Despite that, there are some challenges in adopting LEAN. One challenge in particular, is how complex LEAN's semantics and type system are.

Existing Solutions

When first wanting to formalize a proof in LEAN, one usually takes human mathematicians and sends them to learn the language, and write their proof. This solution comes with the obvious drawback of being slow, requiring a lot of human time and effort. LEAN has a steep learning curve, and it can take months to learn the basics.

```

Goal 2 is the infimum of  $[2, 5)$ .
Proof.
We need to show that  $(2 \text{ is a \_lower bound\_ for } [2, 5)$ 
 $\wedge (\forall l \in \mathbb{R}, l \text{ is a \_lower bound\_ for } [2, 5) \Rightarrow l \leq 2)$ .
We show both statements.
- We need to show that  $(2 \text{ is a lower bound for } [2, 5))$ .
  We need to show that  $(\forall c \in [2, 5), 2 \leq c)$ .
  Take  $c \in [2, 5)$ .
  We conclude that  $(2 \leq c)$ .
- We need to show that
   $(\forall l \in \mathbb{R}, l \text{ is a lower bound for } [2, 5) \Rightarrow l \leq 2)$ .
  Take  $l \in \mathbb{R}$ . Assume that  $(l \text{ is a lower bound for } [2, 5))$ .
  We conclude that  $(l \leq 2)$ .
Qed.

```

Figure 1: A proof written in ROCQ with WATERPROOF.

A more automated approach is to formalize just the theorems and definitions, and then use an automated theorem prover to fill in the gaps. Several such tools exist, such as AESOP, GRIND, and LEANEGG (TODO: cite..?). These tools complete a theorem, or part of a theorem, using various heuristics and algorithms. They tend to be integrated into LEAN, and more fit to use as part of interactive proof development, rather than full automation. Moreover, they tend to work only on simple theorems, fail without human guidance, and even when they succeed, they do not produce readable a proof.

For a more wholistic approach for formalizing a proof, one can reach for any off-the-shelf generative large language model such as CHATGPT. In an agentic setting, these systems have gotten good at generating idiomatic and correct LEAN code. One can write their paper, provide it to the agent, and prompt it to generate a formal LEAN proof. One can even interact with it to steer towards better directions. There are even tools in place to help large language models work with LEAN such ARISTOTLE (TODO: cite?). These systems suffer a number of drawbacks. To name a few: They lack reproducibility, are usually based on cloud services, and are not fit with an interactive and iterative workflow.

There are also libraries for making proofs easier to read and write. One example is WATERPROOF. This is a library for the ROCQ proof assistant, which adds notation inspired by pen-and-paper proofs. An example proof, written with and without WATERPROOF respectively, can be seen in figures 1 and 2. It does require the proof to be fully formalized, and informal proofs would require major work to be adapted to WATERPROOF.

TODO: Mention open ai's paper that adi sent

```

Goal is_infimum 2 (interval_half_open 2 5).
Proof.
  unfold is_infimum.
  split.
  - (* We need to show that 2 is a lower bound for [2, 5) *)
    unfold lower_bound.
    intros c Hc.
    destruct Hc as [H2 _].
    exact H2.
  - (* We need to show that any lower bound l is <= 2 *)
    intros l Hl.
    unfold lower_bound in Hl.
    apply Hl.
    (* Concluding l <= 2 by applying the lower bound to 2 ∈ [2, 5) *)
    split.
    + lra.
    + lra.
Qed.

```

Figure 2: The same proof, written in ROCQ without WATERPROOF.

Our Approach

We define a language for writing *informal* proofs that can be partially translated to LEAN, to help with starting a formal proof. The language is a modification of markdown, with some syntax constructs being recognized as formal. All parts are readable as they are written. The language can be interfaced via LEAN directly by a macro, and can access LEAN and **Mathlib** symbols. It is meant as a way to write an informal proof to be understood, and a formal proof to be checked.

This workflow is in accordance to how researchers tend to use LEAN, as a formal proof in LEAN often comes as a supplementary part of a paper proof inside a research paper.

One difference between our tool and other approaches, is that ours allows an informal proof that is only partially translated. By only aiming to extract as much information as we can from the user, without aiming for a complete rigorous proof, the tool can have less friction both in formalization and in notation.

The tool is invoked by simply calling the macro `define_latex file "foo.md"`, with `foo.md` replaced with the file name of the markdown document of your choice. It is also possible to use the macro with a string literal instead of a file by omitting the "file" keyword: `define_latex "Hello $x=world$"`.

See example in figure 3.

Markdown

Let $H = \{ (x, y), (x + 1, y) \mid x \in 0..8, y \in 0..9 \}$
\$ be the set of horizontal tiles on the board.

Lean

```
def H : Set (Fin 10 × Fin 10) := { (x, y) | (x ∈ Finset.range 9) (y ∈  
Finset.range 10) }
```

Figure 3: Markdown text, and LEAN code that our tool generates

Preliminary Work

We started by using studying a use-case to see what are the important parts in formalization. Then we designed a tool to address those needs, and implemented a proof of concept. We are using a static analysis to choose between different possible representations.

Case Study We choose the following problem as our subject for the initial exploration of the design space:

Can you cover a 10×10 board with 1×2 tiles, such that the number of horizontal and vertical tiles is the same?

As it turns out, you can't. One can prove that it is impossible. A short intuitive proof can be seen in figure 4. While this proof is readable and correct, and even relatively formal, it does not translate well to LEAN. As an example, the figure includes a proof of proposition 3, which states that the horizontal and vertical tiles of the tiling are disjoint.

In general, most lemmas do not translate directly. Logical steps that most mathematicians see as obvious must be stated explicitly, via various different keywords and auxiliary lemmas. Many times, it is hard to find a way to translate the logical steps to LEAN's keywords and `Mathlib`'s lemmas, and there isn't a one to one correspondence.

Another challenge, is the representation of constructs and various mathematical objects. Just in our example, there were various objects that are not obvious to state in a formal way: What is a board? What is a tile? In the proof in figure 4, we choose a specific representation based on sets. But even that is not specific enough to have a single translation to LEAN. One might interpret that as any of the following. Some examples are in figure 5.

Each representation has consequences on how the proof is structured, how hard it will be to formalize, and on the representation of the rest of the proof. The `Set` type is of sets, `Finset` is the type of finite sets. Even the difference between these two representations can affect the entire proof.

#	Assumption	Explanation
1	$H = \{\langle x, y \rangle, \langle x + 1, y \rangle \mid x \in 0..8, y \in 0..9\}$	Set of possible horizontal tiles
2	$V = \{\langle x, y \rangle, \langle x, y + 1 \rangle \mid x \in 0..9, y \in 0..8\}$	Set of possible vertical tiles
3	$D \subseteq H \cup V$ is a valid tiling	
4	$ D \cap H = D \cap V $	Assume for contradiction
	Proposition	Explanation
1	$ D = 50$	Our board's size is 100, and every element is a set of size 2, hence: 50 tiles
2	$(D \cap H) \cup (D \cap V) = D$	By $D \subseteq H \cup V$
3	$(D \cap H) \cap (D \cap V) = \emptyset$	Because H and V are disjoint
4	$ D \cap H = D \cap V = 25$	By the assumption, and by previous two conjectures
5	$\sum_{t \in D \cap V} \sum_{\langle x, y \rangle \in t} x$ is even	Every number in the sum appears twice, as the x 's of a tile's cells are equal
6	$\sum_{t \in D \cap H} \sum_{\langle x, y \rangle \in t} x$ is odd	Here every number appears with its successor by similar reasoning
7	$\sum_{t \in D} \sum_{\langle x, y \rangle \in t} x$ is even	By calculation
8	$\sum_{t \in D} \sum_{\langle x, y \rangle \in t} x = \sum_{t \in D \cap V} \sum_{\langle x, y \rangle \in t} x + \sum_{t \in D \cap H} \sum_{\langle x, y \rangle \in t} x$	By $(D \cap H) \cup (D \cap V) = D$ and $(D \cap H) \cap (D \cap V) = \emptyset$
9	Contradiction	In the previous proposition, the left-hand side is even while the right-hand side is odd

```

lemma disjoint_hvtiling (tiling : DominoTiling n m) :
  Disjoint (htiles tiling) (vtiling tiling) := by
  unfold htiles vtiling
  apply Finset.disjoint_filter_filter'
  apply Pi.disjoint_iff.mpr
  intros
  unfold horizontal vertical
  split <,> simp

```

Figure 4: Top - a proof sketch of the problem, below - a proof of proposition 3 from the table.

```

def Domino (p1 p2 : Point) : Set Point := { p1, p2 }
def Domino (p1 p2 : Point) : Finset Point := { p1, p2 }
inductive Domino
  | H (lft : Point)
  | V (top : Point)
structure Domino where
  topLeft : Point
  isHorizontal : Bool
def Domino n m := { set // isDomino n m set }
where isDomino n m (set : Finset Point) :=
  (∀ p1 p2, p1 ∈ set → p2 ∈ set → p1.distance p2 ≤ 1)
  ∧ set.card = 2
  ∧ (∀ p ∈ set, 0 ≤ p.x < n ∧ 0 ≤ p.y < m)

```

Figure 5: Some possible representations of a domino.

The Tool Our tool takes a markdown file as input and emits definitions directly into the environment of the lean file that calls it. It is used by calling the macro `define_latex file "foo.md"`, with the name of the file instead of `foo.md`. Upon the compiler encountering this macro, it reads the given file and run the following pipeline:

1. Spanning - all math elements (delimited by "\$" or by "\$\$") from the document are extracted.
2. Lexing - math elements are broken into lists of tokens.
3. Paring - each token list is parsed into an Ast based on a grammar for the tool's language.
4. Analyzing - the Ast is translated into a Datalog representation, and a static analysis is run on it. More on this later.
5. Translating - the Ast is translated into LEAN code and added to the files environment via the meta-programming API. More on this later.

In the Lexing and Parsing stages, errors are ignored and their math elements are silently forgotten. This allows the user to write down formulas that will simply be ignored. Ignored errors can optionally be reported as warnings, controlled via a flag.

As a proof of concept, we made a tool which compiles inline math in a markdown document to LEAN code, and uses a static analysis of the document to determine which representation to use for each mathematical object. You can invoke the tool interactively inside a LEAN document, giving it a markdown document, and it declares variables and definitions which can be used from that point on as if they were declared in the file itself directly.

The tool we have right now is very basic, and the only possible representations are `Set` or `Finset` for sets, and `Multiset` for multisets and `Prod` for tuples.

References